Министерство образования Республики Беларусь

Учреждение образования

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина Операционные системы и системное программирование

ОТЧЁТ
к лабораторной работе
на тему

РАБОТА С ФАЙЛАМИ. СИНХРОННЫЙ, АСИНХРОННЫЙ ВВОД-ВЫВОД И ОТОБРАЖЕНИЕ ФАЙЛОВ В ПАМЯТЬ

Вариант 26

Выполнил Д.А. Скробот

Проверил Р.А. Шабанович

Минск 2026

СОДЕРЖАНИЕ

1 Номер и название варианта . . . 3
2 Цель работы . . . 4
3 Краткое описание используемого способа ввода-вывода . . . 5
4 Алгоритм программы . . . 6
4.1 Команда create . . . 6
4.2 Команда add . . . 6
4.3 Команды list и find . . . 6
4.4 Команда generate . . . 6
5 Перечень использованных функций . . . 7
6 Описание структуры файла . . . 8
7 Основные фрагменты исходного кода . . . 9
7.1 Чтение и запись с учётом неполных запросов и EINTR . . . 9
7.2 Little-endian кодирование числа . . . 9
7.3 Обход записей и проверка структуры . . . 9
7.4 Добавление записи с откатом . . . 10
8 Результаты выполнения программы . . . 11
8.1 Таблица экспериментальных данных . . . 11
9 Анализ полученных результатов . . . 13
Заключение . . . 14

1 НОМЕР И НАЗВАНИЕ ВАРИАНТА

Вариант 26: *База данных записей переменной длины*.

Требуется реализовать консольную базу данных записей переменной длины, хранимую в одном бинарном файле. Используется синхронный ввод-вывод на системных вызовах POSIX. В рамках работы необходимо реализовать:

– создание базы данных в бинарном файле с фиксированным заголовком;
– добавление записи со сгенерированным идентификатором UUID;
– просмотр всех записей;
– поиск записи по UUID;
– генерацию тестовой базы заданного объёма.

2 ЦЕЛЬ РАБОТЫ

Изучить механизмы файлового ввода-вывода в операционной системе семейства POSIX, получить практические навыки создания и открытия файлов, блочного чтения и записи данных, позиционирования, определения размера, принудительного сброса данных, обработки ошибок, измерения времени выполнения и исследования влияния способа доступа на производительность.

В рамках работы необходимо:

– проверять корректность входных данных;

– обрабатывать ошибки с выводом названия операции, кода `errno` и текстового описания;

– корректно закрывать созданные файловые дескрипторы;

– не оставлять открытых дескрипторов после завершения программы;

– выводить информацию о ходе работы;

– измерять время выполнения исследуемых операций;

– проверять корректность результата, сравнивая размер и структуру файла с расчётными значениями.

3 КРАТКОЕ ОПИСАНИЕ ИСПОЛЬЗУЕМОГО СПОСОБА ВВОДА-ВЫВОДА

Используется *синхронный* блочный ввод-вывод через файловые дескрипторы. Каждый файл открывается однократно вызовом `open` с соответствующими флагами доступа (`O_RDONLY`, `O_RDWR`, `O_CREAT | O_EXCL`); передача данных выполняется вызовами `read`/`write` блоками малого размера. Для корректной обработки неполных запросов и прерывания по сигналу `EINTR` применяются обёртки `read_exact`/`write_all`, выполняющие запрос до полного переноса данных.

Позиционирование производится с помощью `lseek`, размер файла определяется через `fstat` (поле `st_size`), принудительный сброс данных на носитель — посредством `fsync`. Исключение одновременной записи обеспечивается советующей блокировкой `fcntl` (`F_WRLCK`). Асинхронный ввод-вывод (`aio_*`) и отображение файлов в память (`mmap`) в рамках варианта не применяются. Время измеряется монотонным таймером `clock_gettime(CLOCK_MONOTONIC)`.

Скорость обработки определяется соотношением

$$V = \frac{\text{Size}}{\text{Time}}, \tag{3.1}$$

где Size — объём обработанных данных, Time — время обработки.

4 АЛГОРИТМ ПРОГРАММЫ

Программа поддерживает команды `create <file>`, `add <file> <text>`, `list <file>`, `find <file> <uuid>`, `generate <file> <count> <maxLen>` и `--help`. Коды завершения: `0` — успешное выполнение/«не найдено», `1` — ошибка выполнения или формата, `2` — ошибка аргументов.

4.1 Команда create

Файл открывается с флагами `O_CREAT | O_EXCL | O_WRONLY`; записывается заголовок (версия = 1, count = 0); выполняется `fsync`; заголовок повторно считывается и сверяется. Существующий файл не перезаписывается.

4.2 Команда add

Выполняется проверка текста (ASCII, лимит 1 MiB); генерируется UUID; устанавливается советующая блокировка на запись; структура базы считывается и проверяется целиком; запоминаются прежние размер и count; осуществляется переход в конец файла; дописывается запись; выполняется `fsync`; обновляется count в заголовке; выполняется `fsync`; запись и count повторно считываются и сравниваются с ожидаемыми значениями. При сбое после дописывания производится откат: восстановление count и размера посредством `lseek`/`ftruncate` и `fsync`.

4.3 Команды list и find

Считывается заголовок; записи последовательно обходятся: читается заголовок записи (16 байт UUID и 4 байта длины), затем текст заданной длины; контролируются границы, кодировка ASCII, соответствие счётчика и достижение конца файла. Команда `list` выводит каждую запись, команда `find` сравнивает UUID и возвращает первое совпадение либо сообщение «не найдено». Даже при совпадении обход продолжается для проверки файла целиком.

4.4 Команда generate

Генерируется `count` записей со случайными длинами текста в диапазоне `1..maxLen` и случайными ASCII-символами. Каждая запись повторно считывается и сравнивается побайтово; после чего сохраняется count, выполняется `fsync` и проверка структуры.

5 ПЕРЕЧЕНЬ ИСПОЛЬЗОВАННЫХ ФУНКЦИЙ

Так как работа выполнена на Linux/POSIX API, ниже приведены использованные функции Linux/POSIX и их аналоги в Windows API (см. таблицу 5.1).

Таблица 5.1 – Использованные функции API

Linux/POSIX	Назначение	Аналог Windows API
`open`	создание/открытие файла	`CreateFile`
`read`, `write`	чтение/запись данных	`ReadFile`, `WriteFile`
`lseek`	перемещение позиции	`SetFilePointerEx`
`fstat`	размер файла	`GetFileSizeEx`
`ftruncate`	изменение длины файла	`SetEndOfFile`
`fsync`	сброс данных на носитель	`FlushFileBuffers`
`close`	закрытие дескриптора	`CloseHandle`
`errno`, `strerror`	диагностика ошибок	`GetLastError`, `FormatMessage`
`fcntl`	запрет параллельной записи	`FILE_SHARE_*`
`clock_gettime`	измерение времени	`QueryPerformanceCounter`
`uuid_generate` (libuuid)	генерация UUID	`UuidCreate`
`mmap`	отображение файла в память	`CreateFileMapping`, `MapViewOfFile`
`struct aiocb`, `aio_suspend`	асинхронный ввод-вывод	`OVERLAPPED`

6 ОПИСАНИЕ СТРУКТУРЫ ФАЙЛА

Файл имеет бинарный формат; все числовые поля кодируются в порядке little-endian. Заголовок имеет фиксированный размер 12 байт; за ним следуют записи переменной длины. Сигнатура и контрольная сумма не предусмотрены; размер файла определяется посредством `fstat`. Структура приведена в таблице 6.1.

Таблица 6.1 – Структура файла базы данных

Область	Поля	Размер
Заголовок	`version: uint32` `record_count: uint64`	12 байт

Запись	`uuid: 16 байт`
	`text_length: uint32`
	`text: N байт`
		20 + N байт

UUID хранится в виде 16 байт канонического представления (`libuuid`); текст представлен в кодировке ASCII без завершающего нуля; поле `text_length` определяет число байт текста (максимум 1 MiB). Пустая база имеет размер 12 байт, `version = 1`, `count = 0`. Файл нулевого размера и неверная версия отклоняются.

7 ОСНОВНЫЕ ФРАГМЕНТЫ ИСХОДНОГО КОДА

7.1 Чтение и запись с учётом неполных запросов и EINTR

Листинг 7.1 – Обёртки read_exact и write_all

```
bool readExact(int fd, void* buf, std::size_t len) {
    auto* p = static_cast<char*>(buf);
    std::size_t done = 0;
    while (done < len) {
        const ssize_t n = ::read(fd, p + done, len - done);
        if (n < 0) {
            if (errno == EINTR) continue;
            return false;
        }
        if (n == 0) { errno = 0; return false; }  // преждевременный EOF
        done += static_cast<std::size_t>(n);
    }
    return true;
}

bool writeAll(int fd, const void* buf, std::size_t len) {
    auto* p = static_cast<const char*>(buf);
    std::size_t done = 0;
    while (done < len) {
        const ssize_t n = ::write(fd, p + done, len - done);
        if (n < 0) {
            if (errno == EINTR) continue;
            return false;
        }
        if (n == 0) return false;
        done += static_cast<std::size_t>(n);
    }
    return true;
}
```

7.2 Little-endian кодирование числа

Листинг 7.2 – Функции put_le и get_le

```
void put_le(Bytes& out, std::uint64_t value, int width) {
    for (int i = 0; i < width; ++i) {
        out.push_back(static_cast<std::uint8_t>(value & 0xffu));
        value >>= 8;
    }
}
std::uint64_t get_le(const Bytes& bytes, std::size_t pos, int width) {
    std::uint64_t value = 0;
    for (int i = 0; i < width; ++i)
        value |= static_cast<std::uint64_t>(bytes[pos + i]) << (8 * i);
    return value;
}
```

7.3 Обход записей и проверка структуры

Листинг 7.3 – Функция обхода traverse

```
bool traverse(int fd, std::uint64_t count,
              const std::function<bool(const codec::Record&)>& fn,
              std::string& err) {
```

```
    off_t fsize = 0;
    if (!getSize(fd, fsize)) { err = "failed to get file size"; return
false; }
    std::array<std::uint8_t, kRecordHeader> raw{};
    for (std::uint64_t i = 0; i < count; ++i) {
        if (!readExact(fd, raw.data(), raw.size())) {
            err = "truncated record: missing record header"; return false;
        }
        const std::uint64_t tlen = codec::get_le(bytes(raw), 16, 4);
        if (tlen > codec::kMaxText) { err = "text length exceeds the limit";
return false; }
        const off_t pos = ::lseek(fd, 0, SEEK_CUR);
        if (pos + static_cast<off_t>(tlen) > fsize) {
            err = "truncated record: missing text"; return false;
        }
        codec::Record r;
        std::copy(raw.begin(), raw.begin() + 16, r.uuid.begin());
        r.text.resize(tlen);
        if (tlen > 0 && !readExact(fd, r.text.data(), tlen)) return false;
        if (!codec::isAscii(r.text)) { err = "record text is not ASCII";
return false; }
        if (!fn(r)) return false;
    }
    return ::lseek(fd, 0, SEEK_CUR) == fsize;
}
```

7.4 Добавление записи с откатом

Листинг 7.4 – Фрагмент команды add

```
if (::lseek(fd, 0, SEEK_END) < 0) return errnoMsg("seek", path, errno);
if (!writeAll(fd, recBytes.data(), recBytes.size()))
    return rollbackAdd(fd, oldSize, count);
if (::fsync(fd) != 0) return rollbackAdd(fd, oldSize, count);
const std::uint64_t newCount = count + 1;
if (::lseek(fd, 0, SEEK_SET) < 0) return rollbackAdd(fd, oldSize, count);
writeHeader(fd, kVersion, newCount);
if (::fsync(fd) != 0) return rollbackAdd(fd, oldSize, count);
```

8 РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ ПРОГРАММЫ

Сборка выполнена в среде Linux с флагами `-Wall -Wextra` без предупреждений компилятора. Пример функционирования программы приведён ниже:

```
$ main create demo
created database: demo
$ main add demo "Hello, POSIX!"
added: 97d55c46-41ea-418d-b834-70867ec0f5de  elapsed_ms=1.20
$ main add demo "second record"
added: 2f5c372d-f6ac-4895-a3a0-07dd83bd19f1  elapsed_ms=0.92
$ main add demo ""
added: 029fc8c2-0291-4409-a2c2-52d0a5fb45ec  elapsed_ms=1.15
$ main list demo
97d55c46-41ea-418d-b834-70867ec0f5de  Hello, POSIX!
2f5c372d-f6ac-4895-a3a0-07dd83bd19f1  second record
029fc8c2-0291-4409-a2c2-52d0a5fb45ec
$ main find demo 97d55c46-41ea-418d-b834-70867ec0f5de
97d55c46-41ea-418d-b834-70867ec0f5de  Hello, POSIX!  elapsed_ms=0.01
$ main find demo 00000000-0000-0000-0000-000000000000
not found  elapsed_ms=0.01
$ main create demo        # файл уже существует
exit=1                     # EEXIST, файл не перезаписан
$ main find demo bad-uuid # неверный формат UUID
exit=2
```

Проверка размера для трёх текстов `A`, `Hello` и пустой строки дала значение $12 + 21 + 25 + 20 = 78$ байт, что совпало с расчётным значением, указанным в задании.

8.1 Таблица экспериментальных данных

Эксперимент проводился на сгенерированных базах с максимальной длиной текста `maxLen = 100` байт. Для каждой операции поиска и вставки выполнялось по три замера; приведены три времени и их медиана (см. таблицы 8.1 и 8.2).

Таблица 8.1 – Результаты поиска записи по UUID

count	Размер, байт	maxLen	Операция	t_1, мс	t_2, мс	t_3, мс	Медиана, мс	V, МБ/с
10	736	100	find: найден	0.025	0.029	0.020	0.025	29
10	736	100	find: не найден	0.019	0.019	0.021	0.019	39
100	6930	100	find: найден	0.157	0.109	0.103	0.109	64
100	6930	100	find: не найден	0.103	0.099	0.129	0.103	67
1000	71488	100	find: найден	0.960	0.959	0.982	0.960	74

Продолжение таблицы 8.1

count	Размер, байт	maxLen	Опера-ция	t_1, мс	t_2, мс	t_3, мс	Медиа-на, мс	V, МБ/с
1000	71488	100	find: не найден	1.018	0.953	0.915	0.953	75

Вставка записи в базу из 100 записей (три свежие копии базы):

Таблица 8.2 – Результаты вставки записи

Записей в базе	maxLen	Операция	t_1, мс	t_2, мс	t_3, мс	Медиана, мс
100	100	add	2.100	2.361	1.589	2.100

9 АНАЛИЗ ПОЛУЧЕННЫХ РЕЗУЛЬТАТОВ

Время поиска возрастает практически линейно с увеличением числа записей: для $N = 10, 100, 1000$ медианные значения времени составили примерно 0.025, 0.109 и 0.960 мс соответственно. Наблюдаемая зависимость соответствует линейному обходу всех записей при отсутствии индекса — стоимость поиска прямо пропорциональна объёму базы. Время поиска найденного и отсутствующего идентификатора практически совпадает, поскольку даже при совпадении выполняется полная проверка файла до конца.

Скорость обработки (отношение размера данных ко времени) возрастает с увеличением размера базы (от 29 до 74 МБ/с), что обусловлено амортизацией фиксированных накладных расходов (открытие файла, чтение заголовка) на больший объём данных. Время вставки записи при небольшой базе практически не зависит от числа записей (медиана около 2 мс): операция включает один проход проверки базы, дописывание записи и два вызова `fsync`.

ЗАКЛЮЧЕНИЕ

В результате выполнения работы реализована консольная база данных записей переменной длины на основе системных вызовов POSIX. Программа собрана в среде Linux без предупреждений компилятора; все команды функционируют корректно, а размеры файлов соответствуют расчётным значениям.

Основные результаты:

– поиск по UUID имеет линейную сложность: время возрастает с объёмом базы (от 0.025 до 0.960 мс для N от 10 до 1000 записей);

– размер файла для трёх записей (`A`, `Hello`, пустая строка) равен 78 байт и совпал с расчётным;

– вставка записи при небольшой базе занимает около 2 мс и практически не зависит от числа записей.

Линейный рост стоимости поиска обусловлен следующими причинами:

1 записи имеют переменную длину, поэтому требуется последовательный обход их длин;

2 отсутствует индекс «UUID смещение»;

3 даже при совпадении выполняется полная проверка файла до конца;

4 фиксированные накладные расходы (открытие файла, чтение заголовка) амортизируются на больший объём данных.

Таким образом, при отсутствии индекса поиск по UUID требует полного обхода файла, а стоимость операций определяется как объёмом данных, так и накладными расходами на их обработку.